Summarize

Chapter 15. Organizational Structures

While much of the book so far has focused on the technical challenges in moving toward a fine-grained architecture, we've also looked at the interplay between our microservice architecture and how we organize our teams. In "Toward Stream-Aligned Teams", we looked at the concept of stream-aligned teams, which have end-to-end responsibility for delivery of user-facing functionality, and at how microservices can help make such team structures a reality.

We now need to flesh out these ideas and look at other organizational considerations. As we'll see, if you want to get the most out of microservices, you ignore your company's organization chart at your peril!

Loosely Coupled Organizations

Throughout the book, I've made the case for a loosely coupled architecture and argued that alignment with more autonomous, loosely coupled, stream-aligned teams is likely going to deliver the best outcomes. A shift to a microservice architecture without a shift in organizational structure will blunt the usefulness of microservices—you may end up paying the (considerable) cost for the architectural change, without getting the return on your investment. I've written generally about the need to reduce coordination between teams to help speed up delivery, which in turn enables teams to make more decisions for themselves. These are ideas we'll explore more fully in this chapter, and we'll flesh out some of these organizational and behavioral shifts that are required, but before that, I think it's important to share my vision for what a loosely coupled organization looks like.

In their book *Accelerate*,[1] Nicole Forsgren, Jez Humble, and Gene Kim looked at the characteristics of autonomous, loosely coupled teams to better understand which behaviors are most important to achieving optimum performance. According to the authors, what is key is whether teams can:

- *Make large-scale changes to the design of their system without the permission of somebody outside the team*
- *Make large-scale changes to the design of their system without depending on other teams to make changes in their systems or creating significant work for other teams*
- *Complete their work without communicating and coordinating with people outside their team*
- *Deploy and release their product or service on demand, regardless of other services it depends upon*
- *Do most of their testing on demand, without requiring an integrated test environment*
- *Perform deployments during normal business hours with negligible downtime*

The stream-aligned team, a concept we first encountered in Chapter 1, aligns with this vision for a loosely coupled organization. If you are trying to move toward a stream-aligned team structure, these characteristics make for a fantastic checklist to ensure you are going in the right direction.

Some of these characteristics look more technical in nature—for example, being able to deploy during normal business hours could be enabled through an architecture that supports zero-downtime deployments. But all of these in fact require a behavioral change. To allow teams to have a fuller sense of ownership around their systems, a move away from centralized control is required, including how architectural decision making is done (something we explore in Chapter 16). Fundamentally, achieving loosely coupled organizational structures requires that power and accountability are *decentralized*.

The majority of this chapter will discuss how we make this all work, looking at team sizes, types of ownership models, the role of the platform, and more. There are a host of changes you can consider making to move your organization in the right direction.

Before all that, though, let's explore the interplay between organization and architecture a bit more.

Conway's Law

Our industry is young and seems to be constantly reinventing itself. And yet a few key "laws" have stood the test of time. Moore's law, for example, states that the density of transistors on integrated circuits doubles every two years, and it has proved to be uncan-

nily accurate (although that trend has been slowing). One law that I have found to be almost universally true, and to be far more useful in my day-to-day work, is Conway's law.

Melvin Conway's paper "How Do Committees Invent?," published in *Datamation* magazine in April 1968, observed that:

> *Any organization that designs a system (defined more broadly here than just information systems) will inevitably produce a design whose structure is a copy of the organization's communication structure.*

This statement is often quoted, in various forms, as Conway's law. Eric S. Raymond summarized this phenomenon in *The New Hacker's Dictionary* (MIT Press) by stating, "If you have four groups working on a compiler, you'll get a 4-pass compiler."

Conway's law shows us that a loosely coupled organization results in a loosely coupled architecture (and vice versa), reinforcing the idea that hoping to gain the benefits of a loosely coupled microservice architecture will be problematic without also considering the organization building the software.

Evidence

The story goes that when Melvin Conway submitted his paper on this topic to the *Harvard Business Review*, the magazine rejected it, claiming he hadn't proved his thesis. I've seen his theory borne out in so many different situations that I've accepted it as true. But you don't have to take my word for it: since Conway's original submission, a lot of work has been done in this area. A number of studies have been carried out to explore the interrelation of organizational structure and the systems they create.

Loosely and tightly coupled organizations

In "Exploring the Duality Between Product and Organizational Architectures,"[2] the authors look at a number of different software systems, loosely categorized as being created either by "loosely coupled organizations" or by "tightly coupled organizations." For tightly coupled organizations, think commercial product firms that are typically colocated with strongly aligned visions and goals, while loosely coupled organizations are well represented by distributed open source communities.

In their study, in which they matched similar product pairs from each type of organization, the authors found that the more loosely coupled organizations actually created more modular, less coupled systems, whereas the more tightly coupled organization's software was less modularized.

Windows Vista

Microsoft carried out an empirical study[3] in which it looked at how its own organizational structure impacted the quality of a specific software product, Windows Vista. Specifically, the researchers looked at multiple factors to determine how error-prone a component in the system would be.[4] After looking at multiple metrics, including commonly used software quality metrics like code complexity, they found that the metrics associated with organizational structures (such as the number of engineers who have worked on a piece of code) proved to be the most statistically relevant measures.

So here we have another example of an organization's structure impacting the nature of the system that organization creates.

Netflix and Amazon

Probably the two poster children for the idea that organizations and architecture should be aligned are Amazon and Netflix. Early on, Amazon started to understand the benefits of teams owning the whole life cycle of the systems they managed. It wanted teams to own and operate the systems they looked after, managing the entire life cycle. But Amazon also knew that small teams can work faster than large teams. This led to its infamous *two-pizza teams*, where no team should be so big that it could not be fed by two pizzas. This is, of course, not a totally useful metric—we never find out if we're eating pizza for lunch or dinner (or breakfast!), nor how big the pizzas are—but again, the general idea is that the optimum team size is 8–10 people, and that this team should be customer facing. This driver for small teams owning the whole life cycle of their services is a major reason why Amazon developed Amazon Web Services. It needed to create the tooling to allow its teams to be self-sufficient.

Netflix learned from this example and ensured that from the beginning it structured itself around small, independent teams, so that the services they created would also be independent from each other. This ensured that the architecture of the system was optimized for speed of change. Effectively, Netflix designed the organizational structure for the system architecture it wanted. I've also heard that this extended into the seating plans for teams at Netflix—teams whose services would talk to each other would sit closely together. The idea being that you would want to have more frequent communication with teams that use your services, or whose services you yourself use.

Team Size

Ask any developer how big a team should be, and while you'll get varying answers, there will be general consensus that, to a point, smaller is better. If you push them to put a number on the "ideal" team size, you'll largely get answers in the range of 5 to 10 people.

I did some research into the best team size for software development. I found many studies, but a lot of them are flawed in such a way that drawing conclusions for the broader world of software development is too difficult. The best study I found, "Empirical Findings on Team Size and Productivity in Software Development,"[5] was at least able to draw from a large corpus of data, albeit not one that is necessarily representative of software development as a whole. The researchers' findings indicated that, "as expected from the literature, productivity is worst for those projects with (an average team size) larger or equal to 9 people." This research at least appears to back up my own anecdotal experience.

We like working in small teams, and it's not hard to see why. With a small group of people all focused on the same outcomes, it's easier to stay aligned, and easier to coordinate on work. I do have an (untested) hypothesis that being geographically dispersed or having large time-zone differences among team members will cause challenges that might further limit the optimum team size, but that thinking is likely better explored by someone other than me.

So small teams good, large teams bad. That seems pretty straightforward. Now, if you can do all the work you need to do with a single team, then great! Your world is a simple one, and you could probably skip much of the rest of this chapter. But what if you have more work than you have time for? An obvious reaction to that is to add people. But as we know, adding people may not necessarily help you get more done.

Understanding Conway's Law

Anecdotal and empirical evidence suggests our organizational structure has a strong influence on the nature (and quality) of the systems we create. We also know we want smaller teams too. So how does this understanding help us? Well, fundamentally, if we want a loosely coupled architecture to allow for changes to be made more easily, we want a loosely coupled organization too. Put a different way, the reason we often want a more loosely coupled organization is that we want different parts of the organization to

be able to decide and act more quickly and efficiently, and a loosely coupled systems architecture helps hugely with that.

In *Accelerate*, the authors found a significant correlation between those organizations that had loosely coupled architecture and their ability to more effectively make use of larger delivery teams:

> *If we achieve a loosely coupled, well-encapsulated architecture with an organizational structure to match, two important things happen. First, we can achieve better delivery performance, increasing both tempo and stability while reducing the burnout and the pain of deployment. Second, we can substantially grow the size of our engineering organization and increase productivity linearly—or better than linearly—as we do so.*

Organizationally, the shift that has been coming for a while now, especially for organizations that operate at scale, is a move away from centralized command and control models. With centralized decision making, the speed at which our organization can react is significantly blunted. This is compounded as the organization grows—the bigger it becomes, the more its centralized nature reduces the efficiency of decision making and the speed of action.

Organizations have increasingly recognized that if you want to scale your organization but still want to move quickly, you need to distribute responsibility more effectively, breaking down central decision making and pushing decisions into parts of the organization that can operate with increased autonomy.

The trick, then, is to create large organizations out of smaller, autonomous teams.

Small Teams, Large Organization

> *Adding manpower to a late software project makes it later.*
>
> —Fred Brooks (Brooks's Law)

In his famous essay, "The Mythical Man-Month,"[6] author Fred Brooks attempts to explain why using "a man-month" as an estimation technique is problematic, due to the fact it causes us to fall into the trap of thinking we can throw more people at the problem to go faster. The theory goes like this: if a piece of work will take a developer six months, then if we add a second developer, it will only take three months. If we add five developers, so we now have six in total, the work should be done in just one month! Of course, software doesn't work like that.

For you to be able to throw more people (or teams) at a problem to go faster, the work needs to be able to be split apart into tasks that can be worked on, to some degree, in parallel. If one developer is doing some work that another developer is waiting for, the work cannot be done in parallel, it must be done sequentially. Even if the work can be done in parallel, there is often a need to coordinate between the people doing the different streams of work, resulting in additional overhead. The more intertwined the work, the less effective it is to add more people.

If you cannot break the work apart into subtasks that can be worked on independently, then you cannot just throw people at the problem. Worse, doing so will likely make you slower—adding new people or spinning up new teams has a cost. Time is needed to help these people be fully productive, and often the developers who have too much work to do are the same developers who will need to spend time helping get people up to speed.

The biggest cost to working efficiently at scale in software delivery is the need for coordination. The more coordination between teams working on different tasks, the slower you will be. Amazon, as a company, has recognized this, and structures itself in a way to reduce the need for coordination between its small two-pizza teams. In fact there has been a conscious drive to limit the amount of coordination between teams for this very reason, and restrict this coordination, where possible, to areas where it was absolutely required—between teams that share a boundary between microservices. From *Think Like Amazon*,[7] by ex-Amazon executive John Rossman:

> *The Two-Pizza Team is autonomous. Interaction with other teams is limited, and when it does occur, it is well documented, and interfaces are clearly defined. It owns and is responsible for every aspect of its systems. One of the primary goals is to lower the communications overhead in organizations, including the number of meetings, coordination points, planning, testing, or releases. Teams that are more independent move faster.*

Working out how a team fits into the larger organization is essential. *Team Topologies* defines the concept of a *team API*, which defines broadly how that team interacts with the rest of the organization, not just in terms of microservice interfaces but also in terms of working practices:[8]

> *The team API should explicitly consider usability by other teams. Will other teams find it easy and straightforward to interact with us, or will it be difficult and confusing? How easy will it be for a new team to get on board with our code and working practices? How do we respond to pull requests and suggestions from other teams? Is our team backlog and product roadmap easily visible and understandable by other teams?*

On Autonomy

> *Whatever industry you operate in, it is all about your people, and catching them doing things right, and providing them with the confidence, the motivation, the freedom and desire to achieve their true potential.*
>
> —John Timpson

Having lots of small teams isn't by itself going to help if these teams just become more silos that are still dependent on other teams to get things done. We need to ensure that each of these small teams has the autonomy to do the job it is responsible for. This means we need to give the teams more power to make decisions, and the tools to ensure they can get as much done as possible without having to constantly coordinate work with other teams. So enabling autonomy is key.

Many organizations have shown the benefits of creating autonomous teams. Keeping organizational groups small, allowing them to build close bonds and work effectively together without bringing in too much bureaucracy, has helped many organizations grow and scale more effectively than some of their peers. W. L. Gore and Associates[9] has found great success by making sure none of its business units ever gets to more than 150 people, to make sure that everyone knows each other. For these smaller business units to work, they have to be given power and responsibility to work as autonomous units.

Many of these organizations seem to have drawn from the work done by anthropologist Robin Dunbar, who looked at humans' ability to form social groupings. His theory was that our cognitive capacity places limits on how effectively we can maintain different forms of social relationships. He estimated 150 people as how big a group could grow to before it would need to split off or else it would collapse under its own weight.

Timpson, a highly successful UK retailer, has achieved massive scale by empowering its workforce, reducing the need for central functions, and allowing local stores to make decisions for themselves, such as how much to refund unhappy customers. Now chairman of the company, John Timpson is famous for scrapping internal rules and replacing them with just two:

- Look the part.
- Put money in the till.

Autonomy works at the smaller scale too, and most modern companies I work with are looking to create more autonomous teams within their organizations, often trying to

copy models from other organizations, such as Amazon's two-pizza team model, or the "Spotify model" that popularized the concept of guilds and chapters.[10] I should sound a note of caution here, of course—by all means, learn from what other organizations do, but understand that copying what someone else does and expecting the same results, without actually understanding *why* the other organization does the things it does, may not result in the outcome you want.

If done right, team autonomy can empower people, help them step up and grow, and get the job done faster. When teams own microservices and have full control over those microservices, they can have greater autonomy within a larger organization.

The concept of autonomy starts to shift our understanding of ownership in a microservice architecture. Let's explore this in more detail.

Strong Versus Collective Ownership

In "Defining ownership", we discussed different types of ownership and explored the implications of these styles of ownership in the context of making code changes. As a brief recap, the two primary forms of code ownership we described are:

Strong ownership

: A microservice is owned by a team, and that team decides what changes to make to that microservice. If an external team wants to make changes, it either needs to ask the owning team to make the change on its behalf or else might be required to send a pull request—it would then be entirely up to the owning team to decide under what circumstances the pull request model is accepted. A team may own more than one microservice.

Collective ownership

: Any team can change any microservice. Careful coordination is needed to ensure that teams don't get in each other's way.

Let's further explore the implications of these ownership models and how they can help (or hinder) a drive toward increased team autonomy.

Strong Ownership

With strong ownership, the team owning the microservice calls the shots. At the most basic level, it is in full control of what code changes are made. Taken further, the team

may be able to decide on coding standards, programming idioms, when to deploy the software, what technology is used to build the microservice, the deployment platform, and more. By having more responsibility for the changes that happen to the software, teams with strong ownership will have a higher degree of autonomy, with all the benefits that entails.

Strong ownership is ultimately all about optimizing for that team's autonomy. Coming back to the Amazon way of doing things from *Think Like Amazon*:

> *When it comes to Amazon's famous Two-Pizza Teams, most people miss the point. It's not about the team's size. It's about the team's autonomy, accountability, and entrepreneurial mindset. The Two-Pizza Team is about equipping a small team within an organization to operate independently and with agility.*

Strong ownership models can allow for more local variation. You may be relaxed, for example, about one team deciding to create its microservice in a functional style of Java, as that is a decision that should only impact them. Of course, this variation does need to be tempered somewhat, as some decisions warrant a degree of consistency around them. For example, if everyone else makes use of REST-over-HTTP-based APIs for their microservice endpoints but you decide to use GRPC, you may be causing some issues for other people who want to make use of your microservice. On the other hand, if that GRPC endpoint was only used internally within your team, this might not be problematic. So when making decisions locally that have impact on other teams, coordination may still be needed. Working out when and how to engage a wider organization is something we'll explore shortly when we look at balancing local versus global optimization.

Fundamentally, the stronger the ownership model a team can adopt, the less coordination is needed, and therefore the more productive the team can be.

How far does strong ownership go?

Up until this stage, we've talked primarily about aspects like making code changes or choosing technology. But the concept of ownership can go much deeper. Some organizations adopt a model that I describe as *full life-cycle ownership*. With full life-cycle ownership, a single team comes up with the design, makes the changes, deploys the microservice, manages it in production, and ultimately decommissions the microservice when it is no longer required.

This full life-cycle ownership model further increases the autonomy the team has, as the requirements for external coordination are reduced. Tickets aren't raised with operations

teams to get things deployed, no external parties sign off on changes, and the team decides what changes are to be made and when to ship.

For many of you, such a model may be fanciful, as you already have a number of existing procedures in place regarding how things must be done. You may also not have the right skills in the team to take on full ownership, or you may require new tooling (for example, self-service deployment mechanisms). It's worth noting, of course, that you don't get to full life-cycle ownership overnight, even if you consider it to be an aspirational goal. Don't be surprised if this change can take years to fully embrace, especially with a larger organization. Many aspects of full life-cycle ownership can require cultural changes and significant adjustments in terms of the expectations you have of some of your people, such as the need to support their software outside office hours. But as you can pull more responsibility for aspects of your microservice into your team, you will further increase the autonomy you have.

I don't want to suggest that this model is in any way essential to using microservices—I am a firm believer that strong ownership for multiteam organizations is the most sensible model to get the most out of microservices. A strong ownership model around code changes is a good place to start—you can work to move toward full life-cycle ownership over time.

Collective Ownership

With a collective ownership model, a microservice can be changed by any one of a number of teams. One of the main benefits of collective ownership is that you can move people where they are needed. This can be useful if your bottleneck in terms of delivery is caused by a lack of people. For example, several changes require some updates to be made to the `Payment` microservice to allow for automated monthly billing—you could just assign extra people to implement this change. Of course, throwing people at a problem doesn't always make you go faster, but with collective ownership you certainly do have more flexibility in this regard.

With teams—and people—moving more frequently from microservice to microservice, we require a higher degree of consistency about how things are done. You can't afford a broad range of technology choices or different types of deployment models if you expect a developer to work on a different microservice each week. To get any degree of effectiveness out of a collective ownership model with microservices, you will end up needing to ensure that working on one microservice is much the same as working on any other.

Inherently, this can undermine one of the key benefits of microservices. Coming back to a quote we used at the start of the book, James Lewis has said that "microservices buy you options." With a more collective ownership model, you are likely going to need to *reduce* options in order to introduce a stronger degree of consistency across what teams do and how microservices are implemented.

Collective ownership requires a high degree of coordination among individuals and among the teams those individuals are in. This higher degree of coordination results in an increased degree of coupling at an organizational level. Coming back to the paper by MacCormack et al., mentioned at the beginning of this chapter, we have this observation:

> *In tightly-coupled organizations...even if not an explicit managerial choice, the design naturally becomes more tightly-coupled.*

More coordination can lead to more organizational coupling which in turn leads to more coupled system designs. Microservices work best when they can fully embrace the concept of independent deployability—and a tightly coupled architecture will be the opposite of what you want.

If you have a small number of developers, and perhaps only a single team, a collective ownership model may be totally fine. But as the number of developers increase, the fine-grained coordination needed to make collective ownership work will end up becoming a significant negative factor in terms of getting the benefits of adopting microservice architectures.

At a Team Level Versus an Organizational Level

The concepts of strong and collective ownership can be applied at different levels of an organization. Within a team, you want people to be on the same page, able to efficiently collaborate with each other, and you want to ensure a high degree of collective ownership as a result. As an example, this would manifest itself in terms of all members of a team being able to directly make changes to the codebase. A poly-skilled team focused on the end-to-end delivery of customer-facing software will need to be very good at having collective ownership. At an organizational level, if you want teams to have a high degree of autonomy, then it is important that they also have a strong ownership model.

Balancing Models

Ultimately, the more you tend toward collective ownership, the more important it becomes to have consistency in how things are done. The more your organization tends toward strong ownership, the more you can allow for local optimization, as we see in Figure 15-1. This balance doesn't need to be fixed—you will likely shift this at different times and around different factors. You may, for example, give teams full freedom around choosing a programming language but still require them to deploy onto the same cloud platform for example.



Figure 15-1. The balance between global consistency and local optimization

Fundamentally, though, with a collective ownership model you'll nearly always be forced toward the left end of this spectrum—that is, toward requiring a higher degree of global consistency. In my experience, organizations that get the most out of microservices are constantly trying to find ways to shift the balance more to the right. That said, for the best organizations this isn't something that is set in stone but is more something that is constantly being evaluated.

The reality is that you cannot perform any balancing unless you are aware, to some extent, of what is going on across your organization. Even if you push a great deal of responsibility into the teams themselves, there can still be value in having a function in your delivery organization that can perform this balancing act.

Enabling Teams

We last looked at enabling teams in "Sharing Specialists" in the context of user interfaces, but they have broader applicability than that. As described in *Team Topologies*, these are the teams that work to support our stream-aligned teams. Wherever our microservice-owning, end-to-end focused stream-aligned teams are focusing on delivering user-facing functionality, they need help from others to do their job. When discussing the user interface we talked about the idea of having an enabling team that can help

support other teams in terms of creating effective, consistent user experiences. As shown in Figure 15-2, we can envision this as enabling teams working to support multiple stream-aligned teams in some cross-cutting aspect.

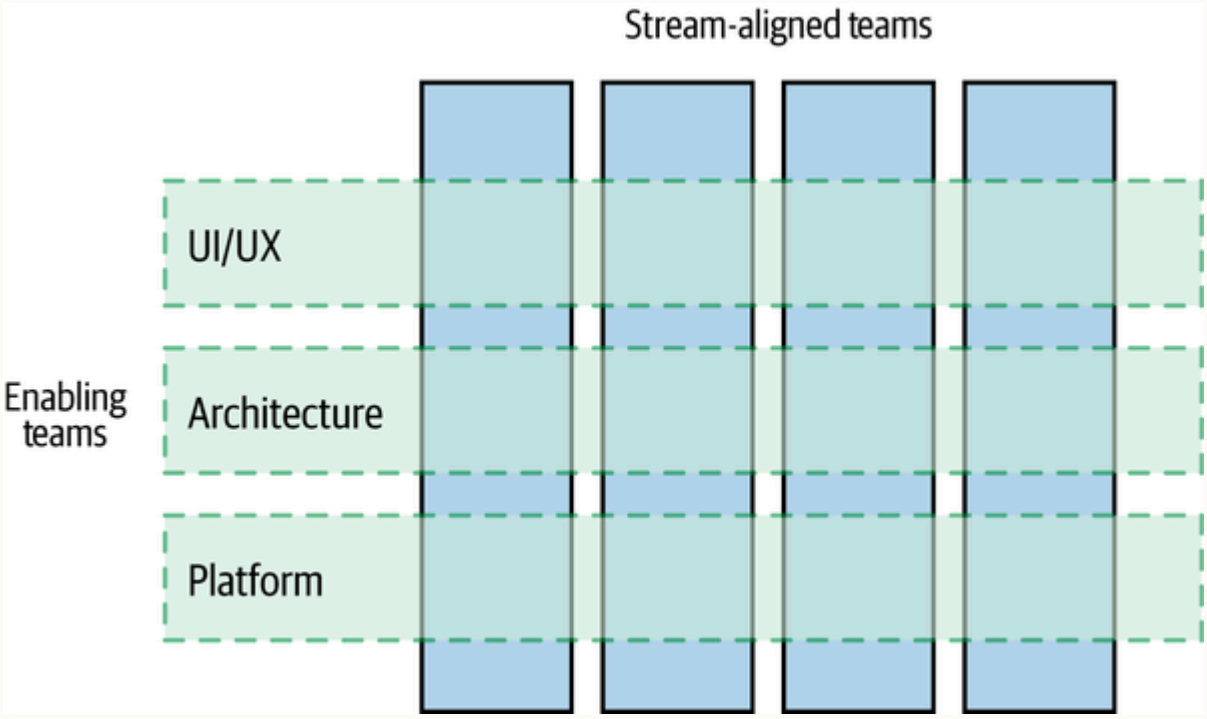


Figure 15-2. Enabling teams support multiple stream-aligned teams

But enabling teams can come in different shapes and sizes.

Consider Figure 15-3. Each team has decided to pick a different programming language. When looked at in isolation, each of these decisions seems to make sense—each team has chosen the programming language that it is happiest with. But what about the organization as a whole? Do you want to have to support multiple different programming languages within your organization? How does that complicate rotation between teams, and how does it affect hiring, for that matter?

You may decide that you actually don't want this much local optimization—but you need to be aware of these different choices being made and have some ability to discuss these changes if you want some degree of control. This is typically where I see a very small supporting group working across the teams to help connect people so these discussions can happen properly.

It's in these cross-cutting supporting groups that I can see the obvious place for architects to be based, at least part of their time. An old-fashioned architect would tell people what to do. In the new, modern, decentralized organization, architects are surveying the landscape, spotting trends, helping connect people, and acting as a sounding board to help other teams get stuff done. They aren't a unit of control in this world; they are yet another enabling function (often with a new name—I've seen terms like *principal en-*

gineer used for people playing the role of what I would consider an architect). We'll explore the role of architects more in Chapter 16.

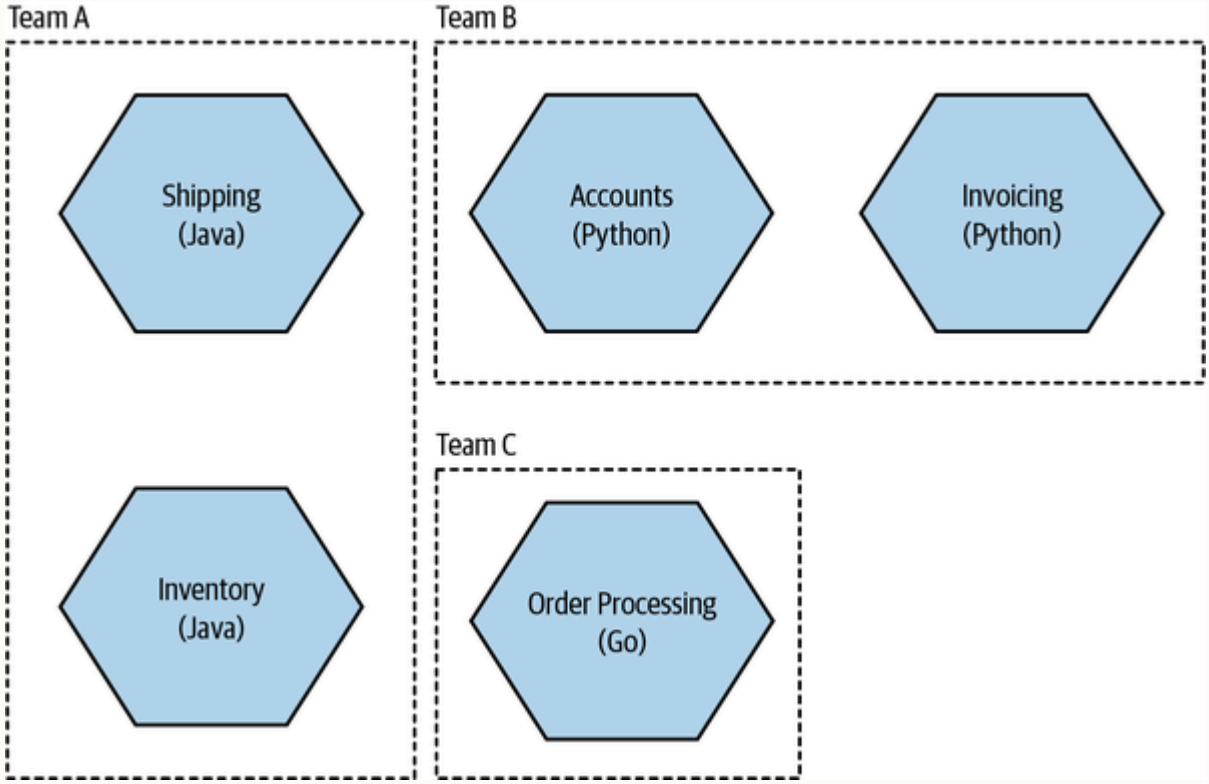


Figure 15-3. Each team has picked a different programming language

An enabling team can help identify problems that would be better fixed outside of the teams as well. Consider a scenario in which each team was finding it painful to spin up databases with test data in them. Each team had worked around this issue in different ways, but the issue was never important enough for any team to fix it properly. But then we look across multiple teams and find that many of them might benefit from a proper solution to the problem, and it suddenly becomes obvious that it needs to be addressed.

Communities of Practice

A *community of practice* (CoP) is a cross-cutting group that fosters sharing and learning between peers. When done well, communities of practice are a fantastic way to create an organization in which people are able to continually learn and grow. In her excellent book on the subject, *Building Successful Communities of Practice*,[11] Emily Webber writes:

> *Communities of practice create the right environment for social learning, experiential learning and a rounded curriculum, leading to accelerated learning for members...It can encourage a learning culture where people seek better ways to do things, rather than only using existing models.*

Now, I believe that in some cases the same group could be both a CoP *and* an enabling team, but in my own experience this is rare. There is definite overlap, though. Both en-

abling teams and communities of practice give you insight into what's happening in different teams across your organization. This insight can help you understand whether you need to rebalance your global versus local optimization, or help you identify the need for some more central help, but the divide here is in the responsibilities and capabilities of the group.

Members of an enabling team are often working full-time as part of the team, or else they have a significant amount of their time ring-fenced for such a purpose. As such, they have more bandwidth to put changes into action—to actually work with other teams and help them. Communities of practice are focused more on enabling learning—the individuals in the group are often participating in a forum for a few hours a week at most, and the membership of such a group is often fluid.

CoPs and enabling teams can work very effectively together, of course. Often, a CoP is able to provide valuable insights that can help an enabling team better understand what is needed. Consider a Kubernetes CoP sharing its experience of how painful it was to work on its company's development cluster with the platform team that manages the cluster. On the subject of platform teams, that is a topic worth looking into in more detail.

The Platform

To return to our loosely coupled, stream-aligned teams, we expect them to do their own testing in isolated environments, manage deployments in such a way that they can be done during the day, and make changes to their system architecture when needed. All of this seems to be pushing more and more responsibility—and work—into these teams. Enabling teams as a general concept can help here, but ultimately stream-aligned teams need a self-service set of tools that allows them to do their job—this is the platform.

Without a platform, in fact, you may find it difficult to change the organization. In his article "Convergence to Kubernetes,"[12] RVU's CTO Paul Ingles shares the experience of price comparison website Uswitch moving away from the direct use of low-level AWS services toward a more highly abstracted platform based on Kubernetes. The idea was that this platform allowed RVU's stream-aligned teams to focus more on delivering new features and spend less time on managing infrastructure. As Paul puts it:

> *We didn't change our organisation because we wanted to use Kubernetes; we used Kubernetes because we wanted to change our organization.*

A platform that can implement common functionality, such as the ability to handle desired state management for microservices, log aggregation, and inter-microservice authorization and authentication, can deliver huge improvements in productivity and enable teams to take on more responsibility without also having to drastically increase the amount of work they do. In fact, a platform should give teams more bandwidth to focus on delivering features.

The platform team

A platform needs someone to run and manage it. These technology stacks can be complex enough to warrant some specific expertise. My concern, though, is that on occasion it can be too easy for platform teams to lose sight of why they exist.

A platform team has users, in the same way that any other team has users. The users of the platform team are other developers—your job, if you are in a platform team, is to make their lives easier (this of course is the job of any enabling team). That means that the platform you create needs to fit the needs of the teams using it. It also means that you need to work with the teams using your platform not only to help them use it well but also to take on board their feedback and requirements to improve the platform you deliver.

In the past, I've preferred calling such a team something like "delivery services" or "delivery support" to better articulate its goal. Really, the job of a platform team *isn't* to build a platform; it's to make developing and shipping functionality easy. Building a platform is just one way that the members of a platform team can achieve this. I do worry that by calling themselves a platform team they will see all problems as things that can and should be solved by the platform, rather than thinking more widely about other ways to make developers' lives easier.

Like any good enabling team, a platform team needs to operate almost like an internal consultancy to some extent. If you are in a platform team, you need to be going out to find out what problems people are facing and working with them to help them get these problems fixed. But as you also end up building the platform too, you need to have a heavy dose of product development work in there as well. In fact, taking a product development approach to how you build out your platform is a great idea and could be a great place to help grow new product owners.

The paved road

A concept that has become popular in software development is that of "the paved road." The idea is that you clearly communicate how you want things to be done and then provide mechanisms whereby these things can be done easily. For example, you might want to ensure that all microservices communicate via mutual TLS. You could then back this up by providing a common framework or deployment platform that would automatically deliver mutual TLS for the microservices running on it. A platform can be a great way to deliver on that paved road.

The key concept behind a paved road is that using the paved road isn't enforced—it just provides an easier way of getting to your destination. So if a team wanted to ensure its microservices communicated via mutual TLS without using the common framework, it would have to find some other way of doing this, but it would still be allowed. The analogy here is that while we might want everyone to get to the same destination, people are free to find their own paths there—the hope is that the paved road is the easiest way of getting where you need.

The paved road concept aims to make the common cases easy, while leaving room for exceptions where warranted.

If we think of the platform as a paved road, by making it optional, you incentivize the platform team to make the platform easy to use. Here is Paul Ingles again, on measuring the effectiveness of the platform team:[13]

> *We'd set [objectives and key results (OKRs)] around the number of teams we'd want to adopt the platform, the number of applications using the platform autoscaling service, the proportion of applications switched to the platform dynamic credentials service, and so on. Some of those we'd track over longer periods of time, and others were helpful to guide progress for a quarter and then we'd drop them in favor of something else.*
>
> *We never mandated the use of the platform, so setting key results for the number of onboarded teams forced us to focus on solving problems that would drive adoption. We also look for natural measures of progress: the proportion of traffic served by the platform, and the proportion of revenue served through platform services are both good examples of that.*

When you place barriers in people's way that seem arbitrary and capricious, those people will find ways of working around the barriers to get a job done. So in general, I find it much more effective to explain why things should be done in a certain way, and then

make it easy to do things in that way, rather than attempting to make the things you don't like impossible.

The shift to more autonomous, stream-aligned teams doesn't eliminate the need for having a clear technical vision or for being clear about certain things that all teams are required to do. If there are concrete restrictions (the need to be cloud-vendor agnostic, for example) or specific requirements that all teams need to obey (all PII needs to be encrypted at rest using specific algorithms), then those still need to be clearly communicated, and the reasons for them made clear. The platform can then play a role in making these things easy to do. By using the platform, you are on the paved road—you'll end up doing a lot of the right things without having to expend much effort.

On the other hand, I see some organizations try to govern via the platform. Rather than clearly articulate what needs to be done and why, instead they simply say "you must use this platform." The problem with this is that if the platform isn't easy to use or doesn't suit a particular use case, people will find ways to bypass the platform itself. When teams work outside the platform, they have no clear sense of what restrictions are important to the organization and will find themselves doing the "wrong" thing without realizing it.

Shared Microservices

As I've already discussed, I'm a big proponent of strong ownership models for microservices. In general, one microservice should be owned by a single team. Despite that, I still find it common for microservices to be owned by multiple teams. Why is this? And what can (or should) you do about it? The drivers that cause people to have microservices shared by multiple teams are important to understand, especially as we may be able to find some compelling alternative models that can address people's underlying concerns.

Too Hard to Split

Obviously, one of the reasons you may find yourself with a microservice owned by more than one team is that the cost of splitting the microservice into pieces that can be owned by different teams is too high, or perhaps your organization might not see the point of it. This is a common occurrence with large, monolithic systems. If this is the main challenge you face, then I hope some of the advice given in Chapter 3 will be of use. You could also consider merging teams to align more closely with the architecture itself.

FinanceCo, a FinTech company that we previously met in "Toward Stream-Aligned Teams", largely operate a strong ownership model with a high degree of team autonomy. However, it still has an existing monolithic system that was slowly being split apart. That monolithic application was to all intents and purposes shared by multiple teams, and the increased cost of working in this shared codebase was obvious.

Cross-Cutting Changes

Much of what we've discussed so far in this book regarding the interplay of organizational structure and architecture is aimed toward reducing the need for coordination between teams. Most of this is about trying to reduce cross-cutting changes as much as we can. We do, though, have to recognize that some cross-cutting changes may be unavoidable.

FinanceCo hit one such issue. When it originally started, one account was tied to one user. As the company grew and took on more business users (being previously more focused on consumers), this became a limitation. The company wanted to move to a model in which a single account with FinanceCo could accommodate multiple users. This was a fundamental change, as up until that point the assumption across the system had been one account = one user.

A single team had been formed to make this change happen. The problem was that a vast amount of the work involved making changes to microservices already owned by other teams. This meant that the team's work was partly about making changes and submitting pull requests, or asking other teams to make the changes happen. Coordinating these changes was very painful, as a significant number of microservices needed to be modified to support the new functionality.

By restructuring our teams and architecture to eliminate one set of cross-cutting changes, we may in fact expose ourselves to a different set of cross-cutting changes that may have a more significant impact. This was the case with FinanceCo—the type of reorganization that would be required to reduce the cost of the multiuser functionality would have increased the cost of making other, more common changes. FinanceCo understood that this particular change was going to be very painful, but that it was such an exceptional type of change that this pain was acceptable.

Delivery Bottlenecks

One key reason people move toward collective ownership, with microservices being shared between teams, is to avoid delivery bottlenecks. What if there is a large backlog

of changes that need to be made in a single service? Let's return to MusicCorp, and let's imagine that we are rolling out the ability for a customer to see the genre of a track across our products, as well as adding a brand new type of stock: virtual musical ring-tones for the mobile phone. The website team needs to make a change to surface the genre information, with the mobile app team working to allow users to browse, preview, and buy the ringtones. Both changes need to be made to the `Catalog` microservice, but unfortunately half the team is stuck diagnosing a production failure, with the other half out with food poisoning after a recent team outing to a pop-up food truck being run out of an alley.

We have a few options we could consider to avoid the need for both the website and mobile teams to share the `Catalog` microservice. The first is to just wait. The website and mobile application teams move on to something else. Depending on how important the feature is or how long the delay is likely to be, this may be fine, or it may be a major problem.

You could instead add people to the catalog team to help them move through their work faster. The more standardized the technology stack and programming idioms in use across your system, the easier it is for other people to make changes in your services. The flip side, of course, as we discussed earlier, is that standardization tends to reduce a team's ability to adopt the right solution for the job and can lead to different sorts of inefficiencies.

Another option that would avoid the need for a shared `Catalog` microservice could be to split the catalog into a separate general music catalog and a ringtone catalog. If the change being made to support ringtones is fairly small, and the likelihood of this being an area in which we will develop heavily in the future is also quite low, this may well be premature. On the other hand, if there are 10 weeks of ringtone-related features stacked up, splitting out the service could make sense, with the mobile team taking ownership.

There are a couple of other models we could consider, however. In a moment, we'll look at what can be done in terms of making the shared microservice more "pluggable," allowing other teams to either contribute their code via libraries or else extend from a common framework. First, though, we should explore the potential to bring some ideas from the world of open source development into our company.

Internal Open Source

Many organizations have decided to implement some form of internal open source, both to help manage the issue of shared codebases, and to make it easier for people outside a team to contribute changes to a microservice they may be making use of.

With normal open source, a small group of people are considered core committers. They are the custodians of the code. If you want a change to an open source project, either you ask one of the committers to make the change for you or you make the change yourself and send them a pull request. The core committers are still in charge of the codebase; they are the owners.

Inside an organization, this pattern can work well too. Perhaps the people who worked on the service originally are no longer on a team together; perhaps they are now scattered across the organization. If they still have commit rights, you can find them and ask for their help, perhaps pairing up with them; or if you have the right tooling, you can send them a pull request.

Role of the Core Committers

We still want our services to be sensible. We want the code to be of decent quality, and the microservice itself to exhibit some sort of consistency in how it is put together. We also want to make sure that changes being made now don't make future planned changes much harder than they need to be. This means that we need to adopt the same patterns used in normal open source internally too, which means separating out trusted committers (the core team) from untrusted committers (people from outside the team submitting changes).

The core ownership team needs to have some way of vetting and approving changes. It needs to make sure changes are idiomatically consistent—that is, that they follow the general coding guidelines of the rest of the codebase. The people doing the vetting are therefore going to have to spend time working with submitters to make sure each change is of sufficient quality.

Good gatekeepers put a lot of work into this, communicating clearly with submitters and encouraging good behavior. Bad gatekeepers can use this as an excuse to exert power over others or have religious wars about arbitrary technical decisions. Having seen both sets of behavior, I can tell you one thing is clear: either way takes time. When considering allowing untrusted committers to submit changes to your codebase, you have to de-

cide if the overhead of being a gatekeeper is worth the trouble: could the core team be doing better things with the time it spends vetting patches?

Maturity

The less stable or mature a service is, the harder it will be to allow people outside the core team to submit patches. Before the key spine of a service is in place, the team may not know what "good" looks like and therefore may struggle to know what a good submission looks like. During this stage, the service itself is undergoing a high degree of change.

Most open source projects tend to not take submissions from a wider group of untrusted committers until the core of the first version is done. Following a similar model for your own organization makes sense. If a service is pretty mature and is rarely changed—for example, our cart service—then perhaps that is the time to open it up for other contributions.

Tooling

To best support an internal open source model, you'll need some tooling in place. The use of a distributed version control tool with the ability for people to submit pull requests (or something similar) is important. Depending on the size of the organization, you may also need tooling to allow for a discussion and evolution of patch requests; this may or may not mean a full-blown code review system, but the ability to comment inline on patches is very useful. Finally, you'll need to make it very easy for a committer to build and deploy your software and make it available for others. Typically this involves having well-defined build and deployment pipelines and centralized artifact repositories. The more standardized your technology stack is, the easier it will be for people in other teams to make edits and provide patches to a microservice.

Pluggable, Modular Microservices

At FinanceCo, I saw an interesting challenge associated with a particular microservice that was becoming a bottleneck for many teams. For each country, FinanceCo had dedicated teams focusing on functionality specific to that country. This made a lot of sense, as each country had specific requirements and challenges. But it caused issues for this central service, which needed specific functionality to be updated for each country. The team owning the central microservice was overwhelmed by pull requests being sent to it. The team was doing an excellent job of processing these pull requests quickly, and in

fact it focused on this being a core part of its responsibilities, but structurally the situation wasn't really sustainable.

This is an example of where a team having a large number of pull requests can be a sign of many different potential issues. Are pull requests from other teams being taken seriously? Or are these pull requests a sign that the microservice should potentially change ownership?

WARNING

If a team has a lot of inbound pull requests, it could be a sign that you really have a microservice that is being shared by multiple teams.

Changing ownership

Sometimes the right thing to do is to change who owns a microservice. Consider an example in MusicCorp. The Customer Engagement team is having to send a large number of pull requests related to the `Recommendation` microservice to the Marketing and Promotions team. This is because that a number of changes are happening as far as how customer information is managed, and also because we need to surface these recommendations in different ways.

In this situation, it may make sense for the Customer Engagement team to just take ownership of the `Recommendation` microservice. In the example of FinanceCo, though, no such option existed. The problem was that the sources of pull requests were coming from *multiple* different teams. So what else could be done?

Run multiple variations

One option we explored was that each country team would run its own variation of the shared microservice. So the US team would run its version, the Singapore team its own variation, and so on.

Of course, the issue with this approach is code duplication. The shared microservice implemented a set of standard behaviors and common rules but also wanted some of this functionality to change for each country. We didn't want to duplicate the common functionality. The idea was for the team that currently managed the shared microservice to instead provide a framework, which really just consisted of the existing microservice with only the common functionality. Each country-specific team could launch its own in-

stance of the skeleton microservice, plugging in its own custom functionality, as shown in Figure 15-4.

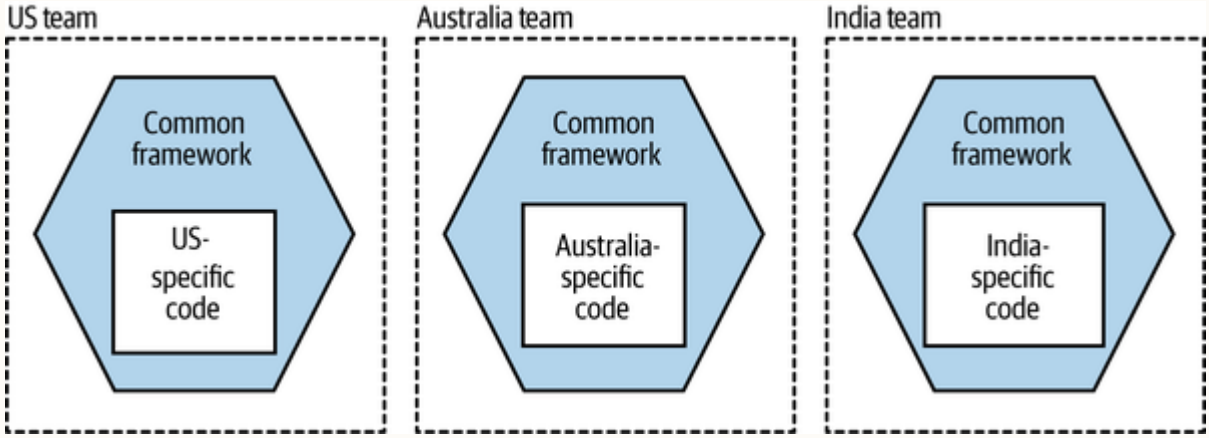


Figure 15-4. A common framework could allow multiple variations of the same microservice to be operated by different teams

The important thing to note here is that although we can share the common functionality in this example across each country-specific variation of the microservice, this common functionality cannot be updated in all variations of the microservice at the same time without requiring a large-scale lockstep release. The core team that manages the framework might make a new version available, but it would be up to each team to pull in the latest version of the common code and redeploy it. In this specific situation, FinanceCo was OK with this restriction.

It is worth highlighting that this specific situation was pretty rare, and something I'd encountered only once or twice before. My initial focus was on finding ways to either split apart the responsibilities of this central shared microservice or else reassign ownership. My concern was that creating internal frameworks can be a fraught activity. It's all too easy for the framework to become overly bloated or to constrain the development of the teams using it. This is the sort of problem that doesn't manifest itself on day one. When creating an internal framework, it all starts with the best intentions. Although in FinanceCo's situation I felt it was the right way forward, I'd caution against adopting this approach too readily unless you've exhausted your other options.

External contribution through libraries

A variation of this approach would be to instead have each country-specific team contribute a library with its country-specific functionality inside it, and then have these libraries packaged together into a single shared microservice, as shown in Figure 15-5.

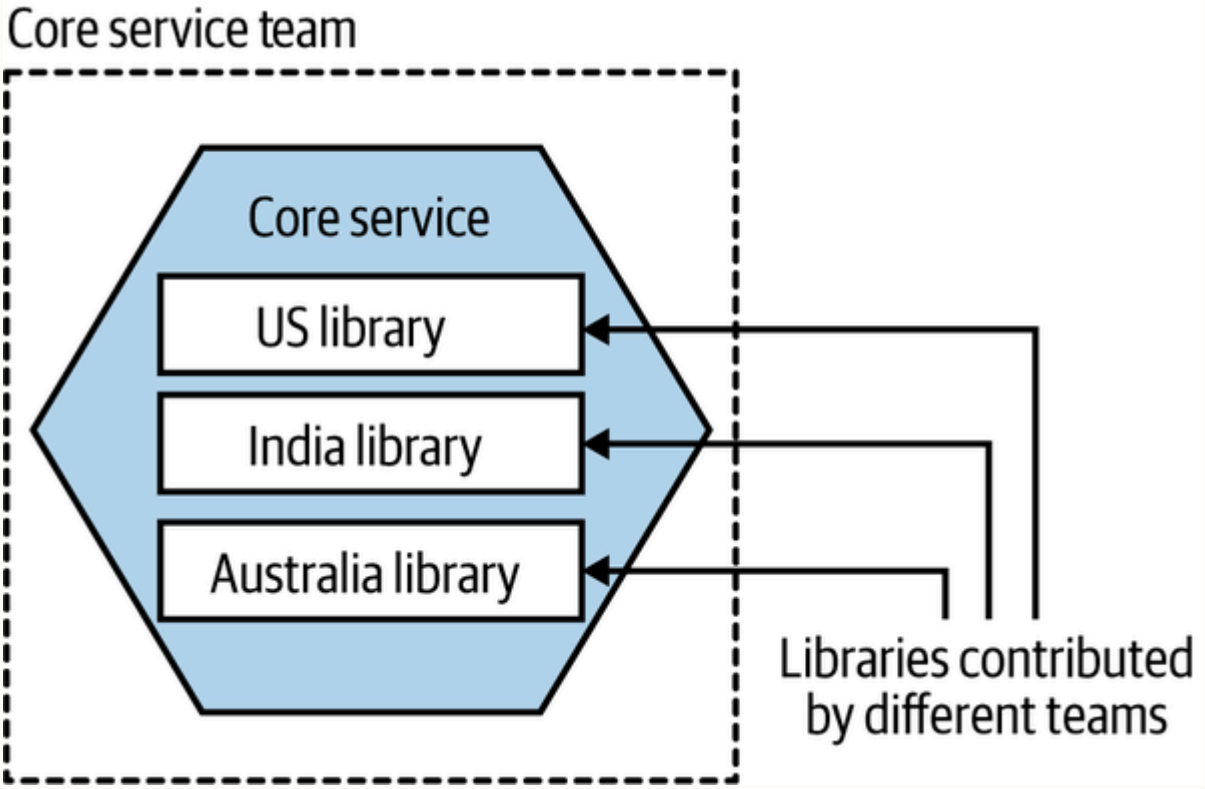


Figure 15-5. Teams contribute libraries with their custom behavior to a central microservice

The idea here is that if the US team needs to implement US-specific logic, it makes a change in a library that is then included as part of the build of the central microservice.

This approach reduces the need to run additional microservices. We don't need to run one service per country—we can run one central microservice that handles the custom functionality for *each* country. The challenge here is that the country teams aren't in charge of deciding when their custom functionality goes live. They can make the change and request that this new change be deployed, but the central team would have to schedule this deployment.

Furthermore, it's possible that a bug in one of these country-specific libraries could cause a production issue that the central team may then be responsible for resolving. This could make production troubleshooting more complex as a result.

Nonetheless, this option might be worth considering if it helps you move away from a central microservice being under collective ownership, especially when you cannot justify running multiple variations of the same microservice.

Change Reviews

When adopting an internal open source approach, the concept of the review is a core principle—the change has to be reviewed before the change can be accepted. But even when working inside a team on a codebase where you have direct commit permissions, there is still value in having your changes reviewed.

I am a big fan of having my changes reviewed. I've always felt that my code has benefited from a second pair of eyes. By far my most preferred form of review is the type of immediate review you get as part of pair programming. You and another developer write the code together and discuss the changes with each other. It gets reviewed before you check in.

You don't have to take my word for it. Coming back to *Accelerate*, a book we've already referenced a number of times now:

> *We found that approval only for high-risk changes was not correlated with software delivery performance. Teams that reported no approval process or used peer review achieved higher software delivery performance. Finally, teams that required approval by an external body achieved lower performance.*

Here we see a distinction drawn between peer-review and external change review. A peer change review is done by someone who is in all likelihood in the same team as you, and who works on the same codebase as you. They are obviously better placed to assess what makes for a good change and are also likely to carry out the review more quickly (more on that shortly). An external review, though, is always more fraught. As the individual is external to your team, they will likely be assessing the change against a list of criteria that may or may not make sense, and as they are in a separate team, they might not get to your changes for a while. As the authors of *Accelerate* note:

> *What are the chances that an external body, not intimately familiar with the internals of a system, can review tens of thousands of lines of code change by potentially hundreds of engineers and accurately determine the impact on a complex production system?*

So, in general, we want to make use of peer change reviews and avoid the need for external code reviews.

Synchronous versus asynchronous code reviews

With pair programming, the code review happens inline at the time the code is being written. In fact, it's more than that. When pairing, you have the driver (the person at the keyboard) and the navigator (who is acting as a second pair of eyes). Both participants are in a constant dialogue about the changes they are making—the act of review and the act of making the changes are happening at the same time. The review becomes an implicit, continual aspect of the pairing relationship. That means that as things are spotted, they are immediately fixed.

If you aren't pairing, the ideal would be for the review to happen very quickly after writing the code. Then you'd like the review itself to be as synchronous as possible. You want to be able to discuss directly with the reviewer any issues they have, agree together on a way forward, make the changes, and move on.

The faster you get feedback on a code change, the faster you can look at the feedback, assess it, ask for clarification, discuss the issue further if needed, and ultimately make any necessary changes. The longer it takes between submitting the code change for review and the review actually happening, the longer and more difficult things become.

If you submit a code change for review and don't get feedback about that change until several days later, you've likely moved on to other work. To process the feedback, you'll need to switch context and re-engage with the work you did previously. You might agree with the reviewer's changes (if required), in which case you can make them and resubmit the change for approval. Worst case, you might need to have a further discussion about the points being raised. This asynchronous back and forth between submitter and reviewer can add *days* to the process of getting changes made.

DO CODE REVIEWS PROMPTLY!

If you want to do code reviews and aren't pair programming, do the review as quickly as possible after the change is submitted, and go through the feedback in as synchronous a way as possible, ideally face to face with the reviewer.

Ensemble programming

Ensemble programming (aka mob programming[14]) is sometimes discussed as a way of doing inline code review. With ensemble programming, a larger group of people (perhaps the whole team) work together on a change. It's primarily about collectively working on a problem and taking input from a large number of individuals.

Of the teams I've spoken to that make use of ensemble programming, most only use it occasionally for specific, tricky problems or important changes, but a lot of development gets done outside the ensemble exercise as well. As such, while the ensemble programming exercise would likely provide sufficient review of the changes being made during the ensemble itself, and in a very synchronous manner at that, you'd still need a way to ensure a review of changes made outside the mob.

Some would argue that you just need to do a review of high-risk changes, and therefore doing reviews only as part of an ensemble is sufficient. It's worth noting that the authors

of *Accelerate* surprisingly found *no* correlation between software delivery performance and reviewing only high-risk changes, compared with a positive correlation when all changes are peer reviewed. So if you do want to ensemble program, go ahead! But you might want to consider reviewing other changes made outside the ensemble too.

Personally, I have some deep reservations around some aspects of ensemble programming. You'll find your team is in fact a neurodiverse bunch, and power imbalances in the ensemble can further undermine the goal of collective problem solving. Not everyone is comfortable working in a group, and an ensemble is definitely that. Some people will thrive in such an environment, while others will feel totally unable to contribute. When I've raised this issue with some proponents of ensemble programming, I've gotten a variety of responses, many of which boil down to the belief that if you create the right ensemble environment, anyone will be able to "come out of their shell and contribute." Let's just say that after that particular set of conversations, I rolled my eyes so much I nearly went blind. To be fair, these same concerns can be raised about pair programming as well!

While I have no doubt that many ensemble programming proponents won't be as unaware or tone-deaf as this, it's important to remember that creating an inclusive workspace is in part about understanding how to create an environment in which all members of the team are able to fully contribute in a way that is safe and comfortable for them. And don't kid yourself that just because you've got everyone in a room everyone is actually contributing. If you'd like some concrete tips on ensemble programming, then I'd suggest reading Maaret Pyhäjärvi's self-published and concise *Ensemble Programming Guidebook*.[15]

The Orphaned Service

So what about services that are no longer being actively maintained? As we move toward finer-grained architectures, the microservices themselves become smaller. One of the advantages of smaller microservices, as we have discussed, is the fact that they are simpler. Simpler microservices with less functionality may not need to change for a while. Consider the humble `Shopping Basket` microservice, which provides some fairly modest capabilities: Add to Basket, Remove from Basket, and so on. It is quite conceivable that this microservice may not have to change for months after first being written, even if active development is still going on. What happens here? Who owns this microservice?

If your team structures are aligned along the bounded contexts of your organization, then even services that are not changed frequently still have a de facto owner. Imagine a

team that is aligned with the consumer web sales context. It might handle the web-based user interface and the `Shopping Basket` and `Recommendation` microservices. Even if the cart service hasn't been changed in months, it would naturally fall to this team to make changes if required. One of the benefits of microservices, of course, is that if the team needs to change the microservice to add a new feature and does not find the feature to its liking, rewriting it shouldn't take too long at all.

That said, if you've adopted a truly polyglot approach and are making use of multiple technology stacks, then the challenges of making changes to an orphaned service could be compounded if your team doesn't know the tech stack any longer.

Case Study: realestate.com.au

For the first edition of this book, I spent some time chatting with realestate.com.au (REA) about its use of microservices, and a lot of what I learned helped greatly in terms of sharing real-world examples of microservices in action. I also found REA's interplay of organizational structure and architecture to be especially fascinating. This overview of its organizational structure is based on our discussions back in 2014.

I am certain that what REA looks like today is quite different. This overview represents a snapshot, a point in time. I'm not suggesting that this is the best way to structure an organization—just that it is what worked best for REA at the time. Learning from other organizations is sensible; copying what they do without understanding why they do it is foolish.

As it does today, REA's core business of real estate encompassed different facets. In 2014, REA was split into independent lines of business (LOBs). For example, one line of business dealt with residential property in Australia, another with commercial property, and another line handled one of REA's overseas businesses. These lines of business had IT delivery teams (or "squads") associated with them; only some had a single squad, while the biggest line had four squads. So for residential property, there were multiple teams involved with creating the website and listing services to allow people to browse property. People rotated between these teams every now and then but tended to stay within that line of business for extended periods, ensuring that the team members could build up a strong awareness of that part of the domain. This in turn helped the communication between the various business stakeholders and the team delivering features for them.

Each squad inside a line of business was expected to own the entire life cycle of each service it created, including building, testing and releasing, supporting, and even decommissioning. The core delivery services team had the job of providing advice, guidance, and tooling to the squads in the LOBs, helping these squads deliver more effectively. Using our newer terminology, the core delivery services team was playing the role of an enabling team. A strong culture of automation was key, and REA made heavy use of AWS as an important part of enabling the teams to be more autonomous. Figure 15-6 illustrates how this all worked.

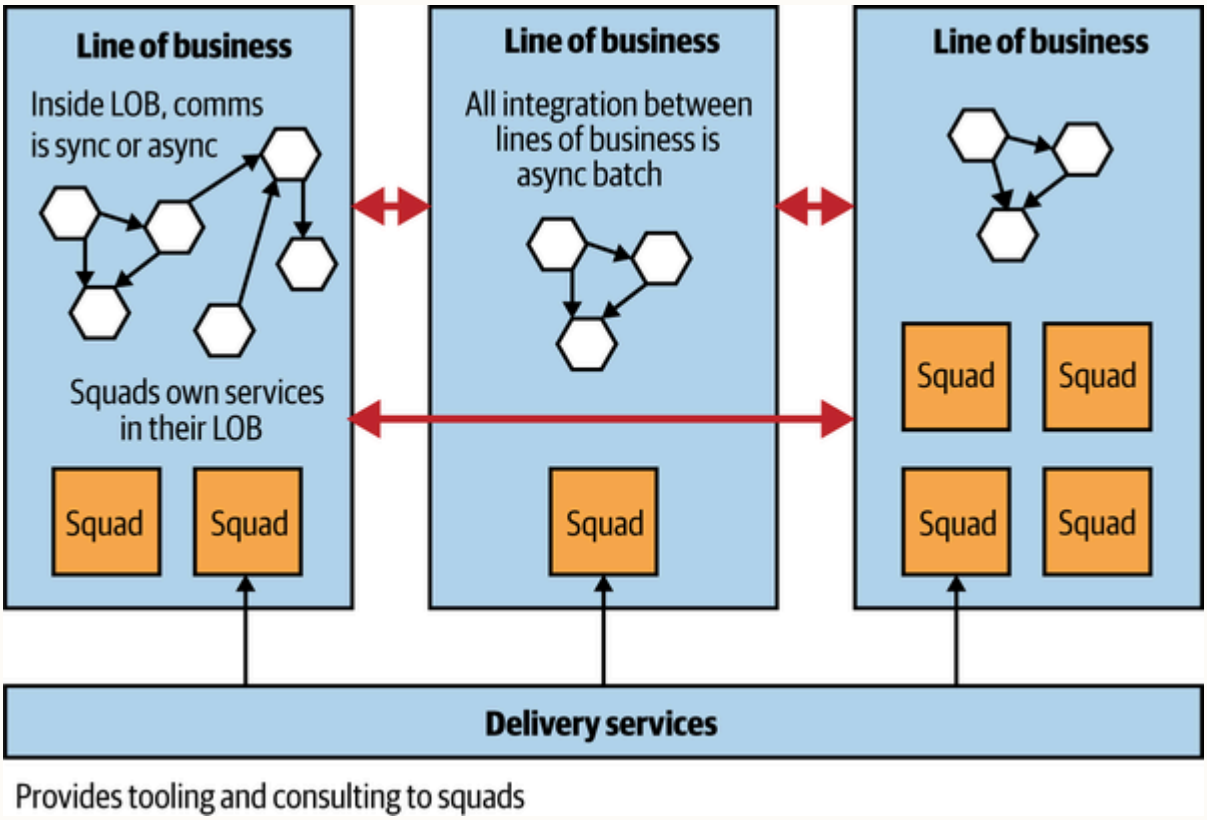


Figure 15-6. An overview of realestate.com.au's organizational and team structure and alignment with architecture

It wasn't just the delivery organization that was aligned to how the business operated. This model extended to the architecture too. One example of this was the integration methods. Within an LOB, all services were free to talk to each other in any way they saw fit, as decided by the squads who act as their custodians. But communication between LOBs was mandated to be asynchronous batch, one of the few cast-iron rules of the very small architecture team. This coarse-grained communication matched that which existed between the different parts of the business too. By insisting on it being batch, each LOB had a lot of freedom in how it acted and how it managed itself. It could afford to take its services down whenever it wanted, knowing that as long as it could satisfy the batch integration with other parts of the business and its own business stakeholders, no one would care.

This structure allowed for significant autonomy not only among the teams but also among the different parts of the business, and the ability to deliver change helped the

company achieve significant success in the local market. This more autonomous structure also helped grow the company from a handful of services in 2010 to hundreds by 2014, facilitating the ability to deliver change faster.

Those organizations that are adaptive enough to change not only their system architecture but also their organizational structure can reap huge benefits in terms of improved autonomy of teams and faster time to market for new features and functionality. REA is just one of a number of organizations that have realized that system architecture doesn't exist in a vacuum.

Geographical Distribution

Colocated teams will find synchronous communication to be very simple, especially as they are typically in the same place at the same time. If your team is distributed, synchronous communication can be more difficult, but it is still achievable if the team members are in the same or similar time zones. When communicating with people in various different time zones, the cost of coordination can increase drastically. In one of my previous roles I worked as an architect, helping support teams based in India, the UK, Brazil, and the US, while I myself was based in Australia. Setting up a meeting between myself and the leads of the various teams was incredibly difficult. This meant that we held these meetings infrequently (typically monthly), and we also had to make sure to discuss only the most important issues during these sessions, as often more than half of the attendees would be working outside their core working hours.

Outside of these sessions, we would have asynchronous communication, largely via email, about other less time-critical issues. But with me in Australia, the delay in this form of communication was significant. I would wake up on a Monday morning to have a pretty quiet start to the week, as most of the world hadn't woken up—this would give me time to process the emails I had received from teams in the UK, Brazil, and the US on their Friday afternoon, which was my Saturday morning.

I recall one client project I worked on in which ownership of a single microservice was shared between two geographical locations. Eventually, each site started specializing the work it handled. This allowed it to take ownership of part of the codebase, within which it could have an easier cost of change. The teams then had more coarse-grained communication about how the two parts interrelated; effectively, the communication pathways made possible within the organizational structure matched the coarse-grained API that formed the boundary between the two halves of the codebase.

So where does this leave us when considering evolving our own service design? Well, I would suggest that geographical boundaries between people involved with the development should be a strong consideration when looking to define both team boundaries and software boundaries. It's much easier for a single team to form when its members are colocated. If colocation isn't possible, and you are looking to form a distributed team, then ensuring that team members are in the same or very similar time zones will aid communication within that team, as it will reduce the need for asynchronous communication.

Perhaps your organization decides that it wants to increase the number of people working on your project by opening up an office in another country. At this point, you should think actively about what parts of your system can be moved over. Perhaps this is what drives your decisions about what functionality to split out next.

It is also worth noting at this point that, at least based on the observations of the authors of the "Exploring the Duality Between Product and Organizational Architectures" report I referenced earlier, if the organization building the system is more loosely coupled (e.g., it consists of geographically distributed teams), the systems being built will tend to be more modular, and therefore hopefully less coupled. The tendency of a single team that owns many services to lean toward tighter integration is very hard to maintain in a more distributed organization.

Conway's Law in Reverse

So far, we've spoken about how the organization impacts the system design. But what about the reverse? Namely, can a system design change the organization? While I haven't been able to find the same quality of evidence to support the idea that Conway's law works in reverse, I've seen it anecdotally.

Probably the best example was a client I worked with many years ago. Back in the days when the web was fairly nascent, and the internet was seen as something that arrived on an AOL floppy disk through the door, this company was a large print firm with a small, modest website. It had a website because that was the thing to do, but in the grand scheme of things the website was fairly unimportant to how the business operated. When the original system was created, a fairly arbitrary technical decision was made as to how the system would work.

The content for this system was sourced in multiple ways, but most of it came from third parties that were placing ads for viewing by the general public. There was an input sys-

tem that allowed content to be created by the paying third parties, a central system that took that data and enriched it in various ways, and an output system that created the final website that the general public could browse.

Whether the original design decisions were right at the time is a conversation for historians, but many years on the company had changed quite a bit, and I and many of my colleagues were starting to wonder if the system design was fit for the company's present state. Its physical print business had diminished significantly, and the revenues and therefore business operations of the organization were now dominated by its online presence.

What we saw at that time was an organization tightly aligned to this three-part system. Three channels or divisions in the IT side of the business aligned with each of the input, core, and output parts of the business. Within those channels, there were separate delivery teams. What I didn't realize at the time was that these organizational structures didn't predate the system design but actually grew up around it. As the print side of the business diminished and the digital side of the business grew, the system design inadvertently laid the path for how the organization grew.

In the end we realized that whatever the shortcomings of the system design were, we would have to make changes to the organizational structure to make a shift. The company is now much changed, but it happened over a period of many years.

People

> *No matter how it looks at first, it's always a people problem.*
>
> —Gerry Weinberg, The Second Law of Consulting

We have to accept that in a microservice environment, it is harder for a developer to think about writing code in their own little world. They have to be more aware of the implications of things like calls across network boundaries or the implications of failure. We've also talked about the ability of microservices to make it easier to try out new technologies, from data stores to languages. But if you're moving from a world in which you have a monolithic system, where the majority of your developers have had to use only one language and have remained completely oblivious to the operational concerns, throwing them into the world of microservices may be a rude awakening for them.

Likewise, pushing power into development teams to increase autonomy can be fraught. People who in the past have thrown work over the wall to someone else are accustomed

to having someone else to blame and may not feel comfortable being fully accountable for their work. You may even find contractual barriers to having your developers carry pagers for the systems they support! These changes can be made in a gradual way, and initially it will make sense to look to change responsibilities for people who are most willing and able to make the change.

Although this book has mostly been about technology, people are not just a side consideration; they are the people who built what you have now and will build what happens next. Coming up with a vision for how things should be done without considering how your current staff will feel about it and without considering what capabilities they have is likely to lead to a bad place.

Each organization has its own set of dynamics around this topic. Understand your staff's appetite for change. Don't push them too fast! Maybe you can have a separate team handle frontline support or deployment for a short period, giving your developers time to adjust to new practices. You may, however, have to accept that you need different sorts of people in your organization to make this all work. You will likely need to change how you hire—in fact, it might be easier to show what is possible by bringing in new people from outside who already have experience in working the way you want. The right new hires may well go a long way toward showing others what is possible.

Whatever your approach, understand that you need to be clear in articulating the responsibilities of your people in a microservices world, and also be clear about why those responsibilities are important to you. This can help you see what your skill gaps might be and think about how to close them. For many, this will be a pretty scary journey. Just remember that without people on board, any change you might want to make could be doomed from the start.

Summary

Conway's law highlights the perils of trying to enforce a system design that doesn't match the organization. This points us, for microservices at least, toward a model where strong ownership of microservices is the norm. Sharing microservices or attempting to practice collective ownership at scale can often result in us undermining the benefits of microservices.

When organization and architecture are not in alignment, we get tension points, as outlined throughout this chapter. By recognizing the link between the two, we'll make sure the system we are trying to build makes sense for the organization we're building it for.

If you would like to explore this topic further, in addition to the previously mentioned *Team Topologies*, I can also thoroughly recommend the talk "Scale, Microservices and Flow," by James Lewis,[16] from which I gained many insights that helped shape this chapter. It is well worth a watch if you are interested in delving more deeply into some of the ideas I have covered in this chapter.

In our next chapter, we will explore in more depth a topic I've already touched on—the role of the architect.

1 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

2 Alan MacCormack, Carliss Baldwin, and John Rusnak, "Exploring the Duality Between Product and Organizational Architectures: A Test of the *Mirroring* Hypothesis," *Research Policy* 41, no. 8 (October 2012): 1309–24.

3 Nachiappan Nagappan, Brendan Murphy, and Victor Basili, "The Influence of Organizational Structure on Software Quality: An Empirical Case Study" *ICSE '08: Proceedings of the 30th International Conference on Software Engineering* (New York: ACM, 2008).

4 And we all know Windows Vista was quite error-prone!

5 Daniel Rodriguez et al., "Empirical Findings on Team Size and Productivity in Software Development," *Journal of Systems and Software* 85, no. 3 (2012), doi.org/10.1016/j.jss.2011.09.009.

6 Frederick P. Brooks Jr, *The Mythical Man-Month: Essays on Software Engineering*, Anniversary ed. (Boston: Addison-Wesley, 1995).

7 John Rossman, *Think Like Amazon: 50 1/2 Ideas to Become a Digital Leader* (New York: McGraw-Hill, 2019).

8 Matthew Skelton and Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).

9 Famous for the development of the waterproof material Gore-Tex.

10 Which famously even Spotify doesn't use anymore.

11 Emily Webber, *Building Successful Communities of Practice* (San Francisco: Blurb, 2016).

12 Paul Ingles, "Convergence to Kubernetes," Medium, June 18, 2018, https://oreil.ly/Ho7kY.

13 "Organizational Evolution for Accelerating Delivery of Comparison Services at Uswitch," Team Topologies, June 24, 2020, *https://oreil.ly/zoyvv*.

14 Although the term *mob programming* is more widespread, I dislike the picture this paints, preferring instead the term *ensemble*, as it makes clear that we have a collection of people working together, rather than a collection of people throwing Molotov cocktails and breaking windows. I'm not 100% sure who came up with this renaming, but I think it was Maaret Pyhäjärvi.

15 Maaret Pyhäjärvi, *Ensemble Programming Guidebook* (self-pub., 2015–2020), *https://ensembleprogramming.xyz*.

16 James Lewis, "Scale, Microservices and Flow," YOW! Conferences, February 10, 2020, YouTube video, 51:03, *https://oreil.ly/ON81J*.

Previous chapter
< 14. User Interfaces

Next chapter
16. The Evolutionary Architect >